

Conceptos de Sistemas Operativos 2012

Procesos IV



- ✓ Versión: Septiembre 2012
- ✓ Palabras Claves: Procesos, Linux, Windows, Creación, Terminación, Fork, Excepción, Relación entre procesos

Algunas diapositivas han sido extraídas de las ofrecidas para docentes desde el libro de Stallings (Sistemas Operativos) y el de Silberschatz (Operating Systems Concepts)



Creación de procesos

- ✓ Un proceso es creado por otro proceso
- ✓ Un proceso padre tiene uno o más procesos hijos.
- ✓ Se forma un árbol de procesos



Actividades en la creación

- ✓ Crear la PCB
- ✓ Asignar PID (Process IDentification) único
- ✓ Asignarle memoria
- ✓ Establecer links de acuerdo a estado, prioridad, etc.
- ✓ Crear estructuras de datos asociadas



Compartir recursos. Alternativas

- ✓ Padres e hijos comparten todos los recursos
- ✓ Proceso hijo comparte un subconjunto de los recursos del padre
- ✓ Proceso Padre e hijo no comparten recursos.



Ejecución. Alternativas

- ✓ Procesos padre e hijo se ejecutan concurrentemente
- ✓ El proceso padre espera hasta que el proceso hijo termine.



Relacion entre procesos Padre e Hijo

Con respecto a los recursos:

- ☑ El hijo consume dentro de los recursos del padre. Comparten los recursos.
- ☑ El hijo gestiona sus propios recursos, independientes de los del padre.



Relacion entre procesos Padre e Hijo (cont)

Con respecto a la Ejecución:

- ✓ El padre continua ejecutándose concurrentemente con su hijo
- ✓ El padre espera que el proceso hijo (o los procesos hijos) terminen para continuar la ejecución.



Relacion entre procesos Padre e Hijo (cont)

Con respecto al Espacio de Direcciones:

- ☑ El hijo es un duplicado del proceso padre (caso Unix)
- ☑ Se crea el proceso y se le carga adentro el programa (caso VMS)



Creación de Procesos

☑ En UNIX:

- ✓ system call **fork()** crea nuevo proceso
- ✓ system call **exec()**, usada después del fork, carga un nuevo programa en el espacio de direcciones.

☑ En Windows:

- ✓ system call **CreateProcess()** crea un nuevo proceso y carga el programa para ejecución.



Análisis de la system call fork

```
...  
int fdold, fdnew;  
...  
fdold = open(argv[1], O_RDONLY);  
...  
fdnew = create(argv[2], 0666); /* create target file rw for all */  
copy(fdold, fdnew);  
...  
}
```

copy(old, new)

```
...  
while ((count = read(old, buffer, sizeof(buffer))) > 0)  
    write(new, buffer, count);  
}
```



Análisis de la system call fork (cont.)

```
if (fork () == 0)
```

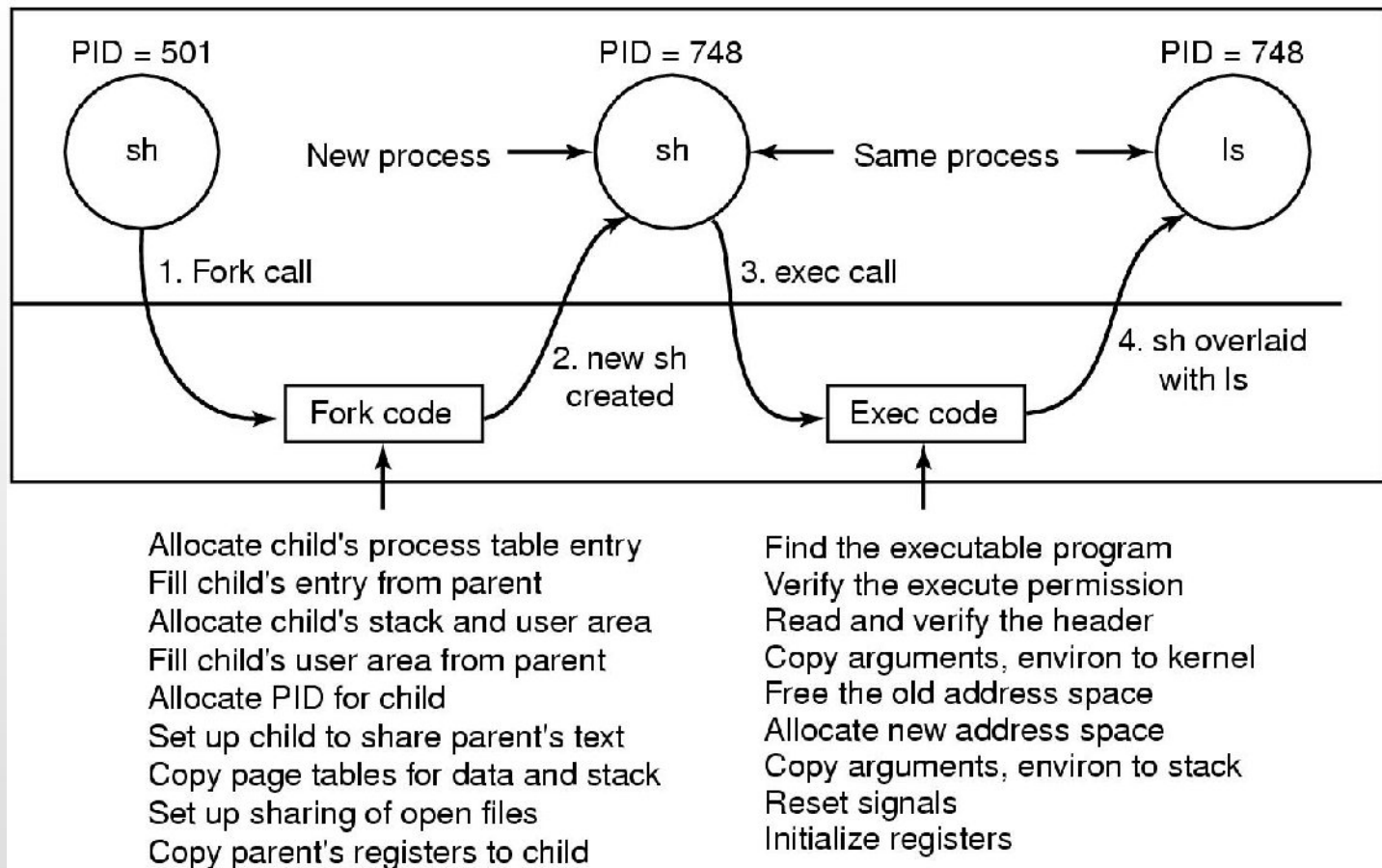
```
    execl("copy", "copy", argv[1], argv[2], 0);
```

```
wait ((int *) 0);
```

```
printf("copy done\n");
```



Fork - Ejemplo



Terminación de procesos

- ☑ Ante un (**exit**), se retorna el control al sistema operativo
 - ✓ EL proceso padre puede recibir un código de retorno (via **wait**)
 - ✓ Los recursos del proceso son liberados
- ☑ Proceso padre puede terminar la ejecución de sus hijos (**kill**)
 - ✓ El proceso hijo se excedió en el uso de los recursos
 - ✓ La tarea asignada al hijo se terminó
 - ✓ Cuando el padre termina su ejecución
 - ◆ Habitualmente no se permite a los hijos continuar, pero existe la opción.
 - ◆ Terminación en cascada



Creación y Terminación de Procesos

☑ Un ejemplo de Shell:

```
while (TRUE) {  
    type_prompt( );  
    read_command (command, parameters)  
  
    if (fork() != 0) {  
        /* Parent code */  
        waitpid( -1, &status, 0);  
    } else {  
        /* Child code */  
        execve (command, parameters, 0);  
    }  
}
```

/* repeat forever */
/* display prompt */
/* input from terminal */

/* fork off child process */

/* wait for child to exit */

/* execute command */



Procesos Cooperativos e Independientes

- ☑ *Independiente*: el proceso no afecta ni puede ser afectado por la ejecución de otros procesos. No comparte ningún tipo de dato.
- ☑ *Cooperativo*: afecta o es afectado por la ejecución de otros procesos en el sistema.



Para que sirven los procesos cooperativos?

- ✓ Para compartir información (por ejemplo, un archivo)
- ✓ Para acelerar el cómputo (separar una tarea en sub-tareas que cooperan ejecutándose paralelamente)
- ✓ Para planificar tareas de manera tal que se puedan ejecutar en paralelo.



System Calls - Unix

System call	Description
<code>pid = fork()</code>	Create a child process identical to the parent
<code>pid = waitpid(pid, &statloc, opts)</code>	Wait for a child to terminate
<code>s = execve(name, argv, envp)</code>	Replace a process' core image
<code>exit(status)</code>	Terminate process execution and return status
<code>s = sigaction(sig, &act, &oldact)</code>	Define action to take on signals
<code>s = sigreturn(&context)</code>	Return from a signal
<code>s = sigprocmask(how, &set, &old)</code>	Examine or change the signal mask
<code>s = sigpending(set)</code>	Get the set of blocked signals
<code>s = sigsuspend(sigmask)</code>	Replace the signal mask and suspend the process
<code>s = kill(pid, sig)</code>	Send a signal to a process
<code>residual = alarm(seconds)</code>	Set the alarm clock
<code>s = pause()</code>	Suspend the caller until the next signal

Syscalls de Procesos



System calls - Windows

Win32 API Function	Description
CreateProcess	Create a new process
CreateThread	Create a new thread in an existing process
CreateFiber	Create a new fiber
ExitProcess	Terminate current process and all its threads
ExitThread	Terminate this thread
ExitFiber	Terminate this fiber
SetPriorityClass	Set the priority class for a process
SetThreadPriority	Set the priority for one thread
CreateSemaphore	Create a new semaphore
CreateMutex	Create a new mutex
OpenSemaphore	Open an existing semaphore
OpenMutex	Open an existing mutex
WaitForSingleObject	Block on a single semaphore, mutex, etc.
WaitForMultipleObjects	Block on a set of objects whose handles are given
PulseEvent	Set an event to signaled then to nonsignaled
ReleaseMutex	Release a mutex to allow another thread to acquire it
ReleaseSemaphore	Increase the semaphore count by 1
EnterCriticalSection	Acquire the lock on a critical section
LeaveCriticalSection	Release the lock on a critical section

Syscalls de Procesos

